

video-edit-agent — User Guide

Version 0.2.2. This is a user-facing guide — for architecture and internals see `agent/core_instructions.md` and the module docstrings under `src/`.

This is the editable Markdown source for the PDF guide (`docs/video-edit-agent-user-guide.pdf`, generated by `scripts/build_user_guide.py`). Edit this file, not the PDF.

What is video-edit-agent?

`video-edit-agent` (CLI: `videoedit`) is an Arabic-first, model-agnostic AI video editing agent. You talk to it in plain language — through Claude Code or directly on the command line — and it turns raw footage, a script, or a folder of scenes into a finished, captioned video. It works fully offline with zero API keys, and is enhanced (never gated) by optional cloud providers (Gemini, ElevenLabs).

The experience is meant to feel like:

[Open Claude Code](#) → [install/open video-edit-agent](#) → [talk naturally](#) → [make videos](#)

not:

[Read technical docs](#) → [memorize CLI commands](#) → [understand package internals](#)

The three workflows: Editor / Creator / Assembler

You don't need to pick a command yourself — describe what you have and what you want, and the agent routes your request. But it helps to know the shape of each:

Editor — you have one raw take

Use when you already have a single recording (talking-head, ad, interview) that needs cleanup and polish. Editor: transcribes it, detects and cuts silence/repetition/false starts, builds an editable EDL, generates Arabic- or English-aware captions (RTL shaping, karaoke word highlighting), plans and places B-roll, applies motion graphics (including behind-subject placement where a usable mask exists), applies your Brand Profile, runs a QA pass, and renders the final video.

CLI: `videoedit edit <video>`

Creator — you have a script or an idea, no footage yet

Use when you're starting from a written script or a described idea. Creator parses the script, plans scenes, analyzes pacing/structure, builds a storyboard, plans the assets you'll need, assembles a MasterTimeline, applies motion treatment, renders, and runs QA.

CLI: `videoedit create <script>`

Assembler — you have several existing scenes/clips

Use when you've already shot multiple scenes and want them combined into one film. Assembler discovers the scenes, preserves your intended order, builds a fast rough cut (`--rough`), then plans continuity and finishes with real transitions, audio crossfade, and loudness normalization (`--finish`) —

reusing the rough cut's plan rather than starting over.

```
CLI: videoedit assemble <scenes_dir> --rough then --finish
```

Feature overview

Editing intelligence — transcript-driven editing; silence, repetition, and false-start handling; an inspectable, re-renderable EDL (change a decision, re-render, no need to redo the whole pipeline); a persistent `project.md` memory explaining what the agent decided and why.

Arabic — RTL captions; Arabic shaping; karaoke-style word highlighting; Egyptian / Gulf / Saudi / MSA preservation; mixed Arabic/English text on the same line; no hidden dialect conversion, ever.

Motion — a built-in simple motion engine that always works offline; optional Remotion rendering for richer motion; automatic engine fallback (logged, never silent); behind-subject compositing status reporting; brand-aware styling.

B-roll — planning with automatic provider fallback; a fully offline path using a local library; optional cloud enhancement (Gemini image/video generation) when API keys are set.

Brand Profiles — reusable colors, fonts, and caption presets per brand, stored as plain folders under `brands/<name>/`; no source changes needed to add one.

Offline / Cloud — see the table below for exactly what needs zero keys versus what cloud providers add. No secrets are ever printed, logged, or written to a config file.

Inspectability — every stage is a plain file you can open directly: `project.md`, the transcript, the EDL, the MasterTimeline, motion/B-roll/QA plans, and the final video.

Arabic-first experience

The single most important rule in this project: **the agent transcribes exactly what was said and never translates, rewrites, formalizes, or dialect-converts spoken Arabic.** Egyptian, Gulf, Saudi, MSA, and code-switched Arabic/English speech are all preserved verbatim. Source audio has final authority.

This is completely independent of the language you use to talk to the agent. If you write to Claude Code in Arabic, the agent's own onboarding/status/explanations/completion reports come back in natural, everyday Arabic — not stiff, over-formal MSA, and not translated from an English template. If you write in English, they come back in English. Either way, your video's transcript and captions are never touched by that choice.

Offline vs. Cloud

Capability	Works with zero API keys	What cloud adds
Transcription	Yes — local Faster-Whisper	Gemini / ElevenLabs transcription (optional, needs API key)
Captions (RTL, shaping, karaoke highlight)	Yes, fully local	—
Editing (silence/repetition/false-start, EDL)	Yes, fully local	—
Motion graphics	Yes — built-in simple engine	Remotion (needs Node/npm, still no API key) for richer motion
B-roll	Yes — local library/planning	Gemini image/video generation for AI-generated B-roll
Visual QA	Yes — basic local checks	Gemini vision for richer QA

Capability	Works with zero API keys	What cloud adds
Brand Profiles, EDL, render, Assembler rough/finish	Yes, fully local	—

GEMINI_API_KEY and ELEVENLABS_API_KEY are read only from environment variables, never from a config file, and never printed or logged by any command — including `videoedit doctor`.

Installing with Claude Code

1. Clone or install this repository into your Claude Code skills directory.
2. Open (or restart) Claude Code.
3. Say: **"Set up video-edit-agent and verify it."**

Claude bootstraps everything: detects your OS and a compatible Python (prefers 3.11), builds a private runtime this project owns (`.runtime/venv` — never your system Python), installs the offline profile, checks `ffmpeg/ffprobe` and `Node/npm`, and reports readiness in plain language, in whichever language you're talking to it in. It never installs OS-level software silently — if `ffmpeg` is missing, it gives you the exact command for your OS and waits for you to run it. No API key is ever required or stored.

First-use examples

Once setup reports ready, just talk to it. Examples:

Arabic → Editor: `videoedit edit my_take.mp4 --brand Arabic --offline`
 Arabic → Creator: `videoedit create my_script.txt --brand Arabic --offline`
 Assembler: `videoedit assemble my_scenes_dir --rough` cut Arabic → other: `videoedit assemble my_scenes_dir --finish --brand Arabic --offline`
 English → Editor: "Edit this video and make it faster and cleaner" English → Creator: "Create a video from this script" English → Assembler: "Assemble these scenes into a rough cut" English → other: "Apply this brand to the captions and motion", "Work offline only"

If your request is genuinely ambiguous the agent asks one short clarifying question rather than guessing.

Editor walkthrough

```
videoedit edit my_take.mp4
```

Output lands in `my_take_edit/` next to the source video: `final.mp4`, `transcript_unified.json`, `edl.json`, `captions.ass`, `project.md`, and QA/B-roll/motion plan JSON files. Use `--brand <name>` to apply a Brand Profile, `--offline` to force zero network calls.

Creator walkthrough

```
videoedit create my_script.txt
```

Produces a storyboard, an asset plan, a MasterTimeline, and a rendered final video, with the same inspectable-artifact pattern as Editor.

Assembler walkthrough

```
videoedit assemble my_scenes_dir --rough
videoedit assemble my_scenes_dir --finish
```

--rough builds a fast preview preserving your scene order. --finish reuses that plan to add real transitions, audio crossfade, and loudness normalization for the final film. You can resume a partially finished project safely.

Brand Profiles

```
videoedit brand init my_brand
videoedit brand validate my_brand
videoedit edit my_take.mp4 --brand my_brand
```

A Brand Profile is a plain folder under brands/<name>/ holding colors, fonts, an avoid-list, a preferred motion engine, and a caption preset.

Outputs and project artifacts

Every pipeline stage is a plain file, never hidden state: project.md (why the agent made each decision), transcript_unified.json, edl.json, captions.ass, motion/B-roll/QA plan JSON files, and final.mp4.

Troubleshooting / Doctor

```
videoedit doctor
```

Prints the full capability matrix (ffmpeg, Node, providers, engines) plus the last videoedit setup run's summary. Run videoedit setup --check for a read-only readiness check, or videoedit setup --repair to force-rebuild the private runtime from scratch without touching anything outside .runtime/.

Known limitations

- Arabic ASR on real spoken audio and Remotion motion rendering are not yet

verified against real acceptance runs in this environment — see the capability status table in the README for the full honesty breakdown.

- J-Cut/L-Cut audio-lead edits are conceptual only, not implemented.
- HyperFrames is not a real installable motion engine; the router always

falls back to a working engine and logs the fallback — never a silent substitution.

Technical appendix: CLI commands

Command	Purpose
videoedit edit <video>	Full pipeline: transcribe → cut → caption → B-roll → motion → render → QA
videoedit create <script>	Script → storyboard → asset plan → motion → render → QA
videoedit assemble <scenes_dir> --rough	Fast rough cut from a folder of pre-shot scenes
videoedit assemble <scenes_dir> --finish	Finish with real transitions + loudness normalization
videoedit doctor	Print the capability matrix
videoedit providers	List transcription/motion providers and availability

Command	Purpose
<code>`videoedit setup [--profile ...] [--check] [--repair]`</code>	Bootstrap the private runtime, check readiness
<code>`videoedit config show / set`</code>	Inspect or edit layered config (never secrets)
<code>`videoedit brand init / validate`</code>	Create or validate a Brand Profile
<code>`videoedit project inspect <dir>`</code>	Print a project's <code>`project.md`</code> memory